

Tech Challenge – Deploy da Aplicação ToggleMaster na AWS

Autor

Júlio Victor Benes Damasceno

1. Objetivo

O objetivo deste Tech Challenge foi implantar a aplicação ToggleMaster em ambiente AWS utilizando uma instância Amazon EC2 para hospedagem da aplicação e um banco Amazon RDS PostgreSQL para persistência dos dados. O trabalho envolveu a configuração da infraestrutura, adaptação da aplicação para utilizar variáveis de ambiente, migração do banco de dados para um serviço gerenciado e validação do funcionamento da API por meio de testes práticos.

2. Tecnologias Utilizadas

Python, Flask, PostgreSQL, Docker, Docker Compose, Gunicorn, Amazon EC2, Amazon RDS PostgreSQL, GitHub, AWS Security Groups, Draw.io e AWS Pricing Calculator.

3. Análise da Arquitetura Monolítica

A ToggleMaster é uma aplicação monolítica porque toda a lógica da aplicação encontra-se concentrada em um único projeto. A API, regras de negócio e acesso ao banco são executados juntos. Essa arquitetura facilita o desenvolvimento de um MVP, reduz a complexidade do deploy e exige menos recursos de infraestrutura. Como desvantagens, apresenta maior acoplamento, limitações para escalabilidade independente dos componentes e necessidade de publicar toda a aplicação mesmo quando apenas uma funcionalidade sofre alteração.

4. Discussão dos 12 Factors

A aplicação atende aos principais fatores do 12-Factor App: utiliza um único repositório Git (Codebase), gerencia dependências pelo requirements.txt (Dependencies), utiliza variáveis de ambiente para configuração (Config), trata o Amazon RDS como serviço externo (Backing Services), executa a aplicação através do Gunicorn (Processes), disponibiliza a aplicação pela porta 5000 (Port Binding) e utiliza Docker Compose para organizar Build e Run. Os fatores relacionados a logs centralizados, alta concorrência, descartabilidade e paridade completa entre desenvolvimento e produção ainda poderiam ser evoluídos em um ambiente corporativo.

5. Arquitetura AWS

A infraestrutura foi composta por uma VPC contendo uma sub-rede pública para a EC2 e uma sub-rede privada para o Amazon RDS PostgreSQL. O acesso externo ocorre através da EC2, enquanto o banco permanece privado e acessível somente pelo Security Group da aplicação.

Essa arquitetura foi escolhida por separar a camada de aplicação da camada de persistência dos dados, aumentando a segurança da solução. Enquanto a aplicação

permanece acessível aos usuários por meio da instância EC2, o banco de dados permanece protegido em uma sub-rede privada, permitindo acesso apenas pela aplicação através dos Security Groups.

https://drive.google.com/file/d/1nxF65WQhUFYISJOR_c4BHPAtsYDt934-/view

6. Infraestrutura

Foi provisionada uma instância Amazon EC2 com Ubuntu Server para hospedar a aplicação. Docker e Docker Compose foram utilizados para construção e execução da API. Um banco Amazon RDS PostgreSQL foi criado e configurado para aceitar conexões exclusivamente da EC2. Após a configuração das variáveis de ambiente, a aplicação passou a utilizar o endpoint do RDS.

7. Segurança

Os Security Groups limitaram o acesso SSH ao IP do administrador. As portas HTTP e 5000 foram disponibilizadas para acesso à aplicação. O Amazon RDS permaneceu protegido, permitindo conexões apenas provenientes do Security Group da EC2.

8. Testes

Foram executados testes utilizando navegador e Postman. O endpoint /health respondeu HTTP 200 e as operações de criação, consulta e atualização de Feature Flags confirmaram o correto funcionamento da aplicação e da integração com o Amazon RDS.

https://www.youtube.com/watch?v=QEtYE_zAONI

9. Estimativa de Custos

A estimativa de custos foi realizada utilizando a AWS Pricing Calculator considerando a infraestrutura implementada durante o projeto. Foram considerados uma instância Amazon EC2 para hospedagem da aplicação e uma instância Amazon RDS PostgreSQL para persistência dos dados, ambas na região **US East (N. Virginia)**. A estimativa permitiu compreender o impacto financeiro da solução e demonstrou que a infraestrutura possui um custo compatível com aplicações de pequeno porte e ambientes de estudo.

<https://drive.google.com/file/d/1isPzzLUgXxazt8Oi-hhSjN0mgsS1822h/view?usp=sharing>

10. Desafios

Durante o desenvolvimento do projeto, o principal desafio foi configurar corretamente a infraestrutura na AWS. A criação do Amazon RDS exigiu ajustes até que a instância fosse compatível com a aplicação, e também foi necessário configurar corretamente os Security Groups para permitir a comunicação entre a EC2 e o banco de dados apenas pela porta 5432.

Outro desafio foi migrar a aplicação do PostgreSQL executado em Docker para o Amazon RDS. Para isso, foi necessário alterar o docker-compose.yaml, configurar as variáveis de ambiente com o endpoint do banco e reconstruir os containers para aplicar as mudanças.

Também foram realizados testes de conectividade utilizando o cliente psql, permitindo validar a comunicação entre a EC2 e o RDS e identificar rapidamente possíveis problemas de configuração. Após esses ajustes, a aplicação foi testada utilizando navegador e Postman, confirmando o funcionamento do endpoint /health e das operações de criação, consulta e atualização das Feature Flags.

Esses desafios contribuíram para o aprendizado sobre infraestrutura em nuvem, segurança de rede, gerenciamento de banco de dados e implantação de aplicações utilizando os serviços da AWS.

11. Conclusão

O projeto atingiu seus objetivos, demonstrando a implantação de uma aplicação monolítica na AWS utilizando serviços gerenciados. A experiência permitiu compreender conceitos de infraestrutura em nuvem, containerização, segurança e integração entre serviços, consolidando os conhecimentos adquiridos durante a disciplina.